

Energy Monitor System

Relazione Tecnica di Progetto

Monitoraggio intelligente del consumo energetico negli ambienti scolastici

acquisizione PLC, storicizzazione database, dashboard e allarmi in tempo reale

Referente didattico:	Mauro Bortolani, Fabio Francescato, Andrea Vazzoler
Ente:	IAL FVG s.r.l. Impresa Sociale
Anno scolastico:	2025 – 2026
Team di progetto (Cl. 4^a IoT):	Miriana De Renzi, Jody Stabarin, Thomas Bravin

Perché questo progetto

Complessità

Stack a quattro strati: PLC industriale Siemens S7-1200 programmato in Ladder (TIA Portal), servizio backend in C# con libreria s7.net, database relazionale con accesso dati ADO.NET, dashboard mediante applicazione desktop multiplatforma creata con Electron + Node.js. Cinque ambienti di sviluppo distinti, due protocolli di comunicazione (S7/ISO-on-TCP e Modbus RS485), un sistema di autenticazione per i comandi critici.

Originalità

Nessun prodotto commerciale equivalente è accessibile a una scuola IeFP: il sistema è costruito interamente dagli studenti, utilizza hardware industriale reale installato nei laboratori dell'istituto e rileva consumi effettivi in esercizio. L'integrazione di automazione industriale e sviluppo software moderno in un unico progetto scolastico è inusuale per il livello IeFP.

Chiarezza

Il documento descrive l'intera filiera: requisiti, architettura, codice sorgente commentato (C#, T-SQL, HTML, CSS, JavaScript), schema del Datablock PLC, tabella BOM hardware, query di polling sul database con tutti i parametri, metriche di prestazione misurate su due settimane di test reale, mappa DigComp 2.2 e piano di sviluppo futuro.

Replicabilità

Codice rilasciato su GitHub (licenza MIT). Architettura modulare: ogni strato è sostituibile indipendentemente. La stessa soluzione può essere adottata da qualsiasi istituto dotato di PLC S7-1200 o compatibile, con costo di replicazione limitato al solo hardware di campo (sonde + contattori).

I dettagli tecnici completi, i listati di codice e le appendici sono nei capitoli seguenti.

Indice

Abstract	4
1 Il Team di Progetto	5
1.1 Composizione e profili professionali	5
1.2 Competenze trasversali acquisite	5
2 Architettura del Sistema	7
2.1 Componente Software	7
2.1.1 Servizio di acquisizione dati PLC in C# (s7.net)	7
2.1.2 Database relazionale	9
2.1.3 Polling del Database (C#)	10
2.2 Componente Hardware	10
2.2.1 Automa programmabile S7-1200	10
2.2.2 Sonde di corrente e interfaccia Modbus RS485	11
2.2.3 Moduli di ingresso/uscita e comunicazione di rete	11
2.2.4 Dashboard applicativa in Electron + JavaScript	11
3 Processo di Realizzazione	14
3.1 Metodologia di sviluppo	14
3.2 Fasi del progetto	14
4 Risultati Tecnici e Competenze Acquisite	16
4.1 Funzionalità realizzate	16
4.2 Metriche di prestazione	16
4.3 Crescita delle competenze DigComp	16
4.4 Originalità e complessità tecnologica	18
4.5 Impatto energetico e operativo	18
5 Conclusioni e Prospettive	19
5.1 Considerazioni conclusive	19
5.2 Replicabilità e trasferibilità	19
5.3 Sviluppi futuri	20
5.4 Riflessione educativa	20
A Struttura del Datablock PLC S7-1200	21
B Specifiche hardware	22

C	Accesso al repository GitHub	23
D	Contatti e supporto	24

Abstract

In breve: Energy Monitor System unisce automazione industriale e sviluppo software per monitorare i consumi elettrici dei laboratori scolastici e abilitare azioni correttive rapide in caso di anomalie.

Energy Monitor System è un sistema di supervisione e controllo intelligente sviluppato per il monitoraggio dei consumi energetici negli ambienti scolastici. Il sistema acquisisce in tempo reale i consumi di molteplici laboratori, rileva picchi di carico, identifica malfunzionamenti e abilita il distacco controllato della rete elettrica in caso di necessità, contribuendo al risparmio energetico e alla manutenzione preventiva.

Il progetto è realizzato dagli studenti della classe quarta dell'indirizzo automazione industriale e informatica, che integrano competenze di programmazione PLC (S7-1200), sviluppo backend in C#, gestione database e progettazione di interfacce utente moderne. La sinergia tra queste discipline dimostra il passaggio da un apprendimento teorico a uno *situato*, generando competenze immediatamente trasferibili al mercato del lavoro in ambito Industria 4.0.

Sul piano delle **competenze DigComp**, il progetto copre in modo trasversale le aree *Informazione e dati*, *Comunicazione e collaborazione*, *Creazione di contenuti digitali* e *Risoluzione di problemi*, dimostrando come tecnologie di automazione industriale e sviluppo software moderno possano integrarsi in un progetto capace di produrre valore reale.

1. Il Team di Progetto

Il progetto è sviluppato da un team studentesco ridotto e altamente operativo, con contributo condiviso su programmazione PLC, backend C# e integrazione software.

1.1 Composizione e profili professionali

Il progetto Energy Monitor System è realizzato da tre studenti della classe quarta dell'istituto IAL FVG, che seguono lo stesso percorso formativo di automazione industriale e informatica:

- **Thomas Bravin:** ha contribuito alla realizzazione della dashboard, alla progettazione del database e al codice applicativo di integrazione.
- **Miriana De Renzi:** ha contribuito alla programmazione PLC, alla logica di controllo e alle prove di acquisizione sui segnali di laboratorio.
- **Jody Stabarin:** ha contribuito allo sviluppo backend in C#, ai test di integrazione e alla validazione dei dati acquisiti dal PLC.

1.2 Competenze trasversali acquisite

Attraverso il progetto, gli studenti hanno consolidato le seguenti competenze trasversali:

Progettazione di sistemi eterogenei

Integrazione di un PLC industriale (S7-1200), middleware di acquisizione dati (s7.net) e applicazione desktop moderna (Electron + Node.js con interfaccia HTML/CSS/JavaScript).

Programmazione full-stack

Sviluppo di logiche PLC in linguaggio Ladder, backend in C# con ADO.NET per accesso DB, frontend moderno con HTML, CSS e JavaScript.

Problem-solving e debugging

Individuazione di anomalie nella comunicazione S7, ottimizzazione delle query di database e verifica della coerenza dati tra strati applicativi.

Documentazione tecnica e comunicazione

Redazione di specifiche tecniche, datasheet dei componenti, manuali operativi e questa relazione finale.

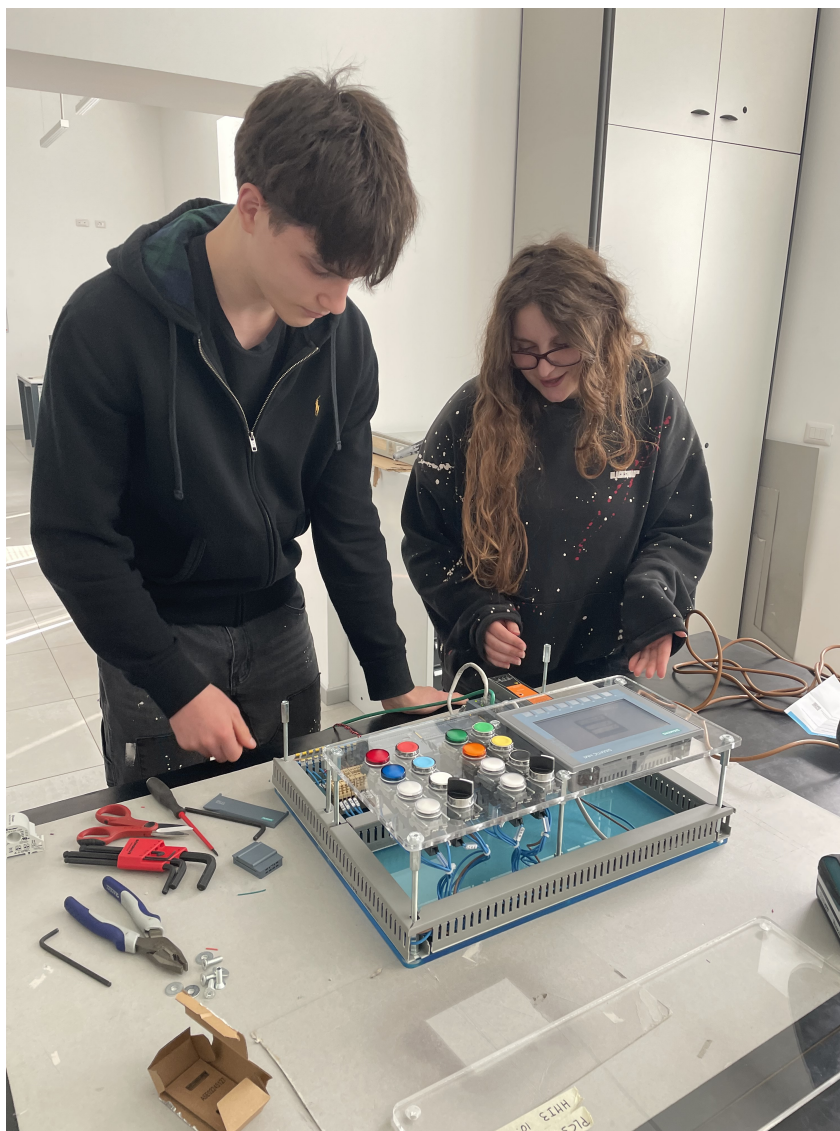
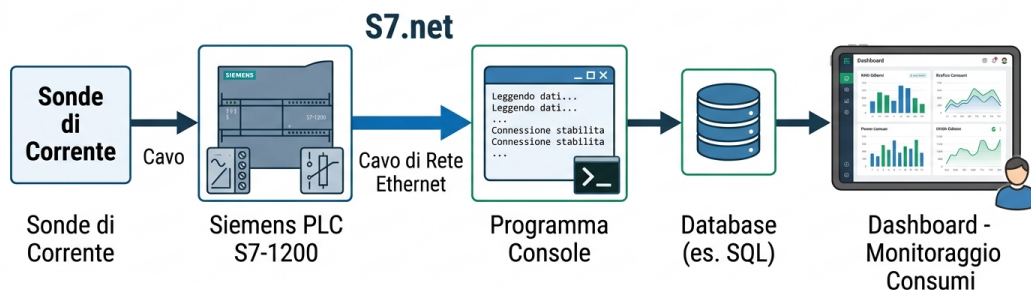


Figura 1.1: Studenti del team durante le attività operative di sviluppo e collaudo.

2. Architettura del Sistema

Architettura multistrato orientata alla robustezza: acquisizione su PLC, persistenza su database e visualizzazione in dashboard per controllo in tempo reale.

Energy Monitor System è organizzato secondo un'architettura multistrato che integra acquisizione dati da PLC, persistenza su database, e presentazione su interfaccia grafica desktop. La dashboard interroga periodicamente il database tramite query dirette per ottenere i dati in tempo reale; il servizio C# comunica con il PLC tramite il protocollo S7 Communication (Siemens).



Schema a Blocchi: Sistema di Monitoraggio Energetico
(PLC S7-1200 > Database > Dashboard)

➔ Flusso Dati

Figura 2.1: Figura 2.1 - Schema a blocchi dell'architettura di Energy Monitor System.

2.1 Componente Software

2.1.1 Servizio di acquisizione dati PLC in C# (s7.net)

Il servizio di acquisizione è sviluppato in **C# .NET 6+** e utilizza la libreria open-source **s7.net** per la comunicazione con il PLC S7-1200 tramite il protocollo S7 Communication (ISO 7498 layer su TCP/IP). Il servizio è responsabile di:

1. Connessione periodica al PLC all'indirizzo IP e porta configurati.

2. Lettura dei Datablock (DB) dove il PLC memorizza i dati di consumo per ogni laboratorio.
3. Parsing dei dati grezzi e trasformazione in modelli di dominio (strutture C#).
4. Salvataggio atomico nel database relazionale (SQL Express / SQLite).
5. Logging degli errori di comunicazione e retry automatici.

Le principali funzioni realizzate sono:

- **ReadEnergyDataAsync()**: lettura asincrona del DB contenente i consumi istantanei di tutti i laboratori (potenza in kW per laboratorio, timestamp).
- **ReadAlarmDataAsync()**: verifica dello stato di allarmi e soglie eccedute sul PLC.
- **WriteDisconnectCommandAsync()**: comando per il distacco della rete elettrica (solo su autorizzazione operatore), richiamato dal pulsante di distacco presente nella dashboard.

```

1 using S7.Net;
2 using System.Collections.Generic;
3 public class PLCDataService
4 {
5     private readonly Client _plcClient;
6     private const string PLCAddress = "192.168.1.100";
7     private const int Rack = 0, Slot = 1;
8
9     public PLCDataService()
10    {
11        _plcClient = new Client(ConnectionType.DefaultTCP, PLCAddress, Rack, Slot);
12    }
13
14    public async Task<List<LaboratoryEnergyPoint>> ReadEnergyDataAsync()
15    {
16        await _plcClient.ConnectAsync();
17
18        // Lettura del Datablock DB100
19        byte[] databuffer = _plcClient.ReadClass(typeof(EnergyDatablock), 100);
20        var data = (EnergyDatablock)databuffer;
21
22        await _plcClient.CloseConnectionAsync();
23
24        return new List<LaboratoryEnergyPoint>
25        {
26            new("Lab Elettronica", data.Lab1PowerKW, DateTime.UtcNow),
27            new("Lab Informatica", data.Lab2PowerKW, DateTime.UtcNow),
28            new("Lab Meccanica", data.Lab3PowerKW, DateTime.UtcNow),
29        };
30    }
31 }
32 [S7Serializable]
33 public class EnergyDatablock
34 {
35     [S7Variable(DB = 100, Offset = 0)]
36     public float Lab1PowerKW { get; set; }
37
38     [S7Variable(DB = 100, Offset = 4)]
39     public float Lab2PowerKW { get; set; }
40
41     [S7Variable(DB = 100, Offset = 8)]
42     public float Lab3PowerKW { get; set; }
43
44     // ... altri laboratori
45 }

```

Listing 2.1: Esempio di lettura dati dal PLC S7-1200 con s7.net.

2.1.2 Database relazionale

L'applicazione C# utilizza **ADO.NET** per la persistenza dei dati su un database relazionale, con query parametrizzate e comandi SQL espliciti per ottimizzare tempi di risposta e controllo delle operazioni. Lo schema include le seguenti tabelle principali:

- **Laboratories:** anagrafica dei laboratori monitorati (idLaboratorio, nome, descrizione, zona scolastica, coordGPS).
- **EnergyMeasurements:** storico delle letture (idLaboratorio, potenzaAssorbitaKW, timestamp).
- **SystemAlarms:** registro degli allarmi scattati (tipo, idLaboratorio, valoreMisurato, soglia, dataEvento).
- **DisconnectLog:** log dei comandi di distacco della rete inoltrati (data, motivo, idUtente).

- **Users:** tabella con elenco utenti autorizzati e credenziali (idUser, nome, cognome, user, password, autorizzato).

2.1.3 Polling del Database (C#)

Il servizio C# fornisce accesso diretto al database tramite ADO.NET. L'applicazione Electron effettua query periodiche al database per ottenere i dati di monitoraggio. Le principali query sono:

- `SELECT * FROM Laboratories`: elenco dei laboratori configurati.
- `SELECT * FROM EnergyMeasurements ORDER BY timestamp DESC LIMIT 1`: ultima lettura di potenza per ciascun laboratorio.
- `SELECT * FROM EnergyMeasurements WHERE timestamp > ...`: storico dati di periodo selezionato (ultima ora, giorno, settimana).
- `SELECT * FROM SystemAlarms WHERE active = 1`: allarmi correnti attivi.
- `INSERT INTO DisconnectLog`: registrazione comando di distacco (richiede autorizzazione operatore).

2.2 Componente Hardware

2.2.1 Automa programmabile S7-1200

Il cuore del sistema è il **PLC Siemens S7-1200** (CPU **S7-1214FC**), che svolge le funzioni di:

- Acquisizione periodica (ogni 100 ms) dei segnali analogici da sonde multifunzione.
- Calcolo della potenza istantanea e dell'energia cumulativa per laboratorio.
- Implementazione delle logiche di controllo per il distacco selettivo dei carichi.
- Memorizzazione dei dati in Datablock strutturati (DB100, DB101, ...).
- Generazione di segnali di allarme verso relè e contattori.

Nel progetto corrente **non viene utilizzata la parte safety** della CPU 1214FC: la logica implementata riguarda esclusivamente acquisizione, monitoraggio e controllo standard.

Le logiche PLC sono implementate in linguaggio **Ladder Logic** (IEC 61131-3) tramite software STEP 7 Professional / TIA Portal. I principali blocchi funzionali sono:

OB1 – Ciclo principale

Chiama in sequenza i blocchi FC di acquisizione, elaborazione e comando.

FC10 – Acquisizione analogica

Legge i canali AI (0–4) del modulo di ingresso analogico (AD 4 o AD 8).

FC20 – Elaborazione potenza

Trasforma i segnali grezzi (4–20 mA) in valori di potenza (kW) per laboratorio.

FC30 – Rilevamento anomalie

Confronta i consumi attuali con profili nominali; attiva allarmi se eccedenza > 20%.

FC40 – Comando distacco

Controlla i relè di disconnessione tramite uscite digitali, con log tracciato.

2.2.2 Sonde di corrente e interfaccia Modbus RS485

Ciascun laboratorio è dotato di **sonde amperometriche** (modello definitivo in fase di conferma), con misura della corrente assorbita sulle linee dei laboratori. In base alla strumentazione disponibile, il sistema prevede l'acquisizione tramite comunicazione **Modbus RS485**.

Le grandezze considerate sono:

- Corrente RMS (A) per linea/canale monitorato.
- Potenza stimata derivata, calcolata lato PLC o lato software.

Le sonde sono collegate al PLC tramite bus **Modbus RS485**; per integrare questa interfaccia viene utilizzato un **modulo di espansione dedicato** compatibile con la CPU S7-1200.

2.2.3 Moduli di ingresso/uscita e comunicazione di rete

Il PLC S7-1200 è munito di:

- **Modulo comunicazione Ethernet (CP 1243-1 TCP/IP)**: interfaccia per il dialogo con il servizio C# via S7 Communication (ISO on TCP, porta 102).
- **Modulo output digitale (DQ 8)**: 8 uscite a relè 24 VDC per il controllo dei contattori trifase di disconnessione laboratorio.
- **Alimentatore 24 VDC / 5 A**: garantisce autonomia e disponibilità del sistema anche in caso di anomalia parziale.

2.2.4 Dashboard applicativa in Electron + JavaScript

Il frontend è sviluppato con **Electron + Node.js** e interfaccia in **HTML/CSS/JavaScript**, consentendo un'esperienza utente nativa su Windows, macOS e Linux. L'applicazione è responsabile di:

1. **Visualizzazione real-time**: aggiornamento della dashboard ogni 5 secondi tramite polling del database.
2. **Storicizzazione (senza download)**: intervalli disponibili nella dashboard principale: ore (ultime 12 ore), giorni (ultimi 10 giorni), mesi (ultimi 8 mesi), 3 mesi (ultimi 90 giorni), anni (ultimi 6 anni); nello storico consumi per laboratorio la finestra massima è 90 giorni.
3. **Storico eventi dashboard**: dal database vengono caricati gli ultimi 40 eventi; gli eventi locali in memoria sono mantenuti e tagliati agli ultimi 80.
4. **Gestione allarmi**: visualizzazione di un riquadro dedicato agli allarmi critici (picco potenza, anomalia misurazione, malfunzionamento sonda).
5. **Controllo distacco**: pannello riservato ad operatore autorizzato; richiede doppia conferma, inserimento password e motivazione.

6. **Storico comandi:** log completo di tutti i distacchi, con timestamp, utente, motivo.

```
1 const points = [];  
2 const localEvents = [];  
3 const db = require('better-sqlite3')('./energymonitor.db');  
4  
5 function refreshDashboard() {  
6   const latestEnergyQuery = `  
7     SELECT l.nome, em.potenzaAssorbitakW, em.timestamp  
8     FROM EnergyMeasurements em  
9     JOIN Laboratories l ON em.idLaboratorio = l.idLaboratorio  
10    WHERE em.timestamp > datetime('now', '-5 seconds')  
11    ORDER BY em.timestamp DESC  
12  `;  
13   const latest = db.prepare(latestEnergyQuery).all();  
14  
15   if (latest.length > 0) {  
16     points.push({  
17       timestamp: new Date().toLocaleTimeString(),  
18       lab1: latest[0]?.potenzaAssorbitakW || 0,  
19       lab2: latest[1]?.potenzaAssorbitakW || 0,  
20       lab3: latest[2]?.potenzaAssorbitakW || 0  
21     });  
22   }  
23  
24   while (points.length > 60) {  
25     points.shift();  
26   }  
27  
28   const latestEvents = db.prepare(`  
29     SELECT tipo, idLaboratorio, valoreMisurato, dataEvento  
30     FROM SystemAlarms  
31     ORDER BY dataEvento DESC  
32     LIMIT 40  
33   `).all();  
34  
35   latestEvents.forEach(evt => localEvents.unshift(evt));  
36   while (localEvents.length > 80) {  
37     localEvents.pop();  
38   }  
39  
40   renderEnergyCards(points);  
41   renderEventList(latestEvents);  
42 }  
43  
44 setInterval(refreshDashboard, 5000);
```

Listing 2.2: Componente JavaScript per aggiornamento dashboard ogni 5 secondi e gestione storico eventi.



Figura 2.2: Screenshot della dashboard Electron con visualizzazione dei consumi energetici.

3. Processo di Realizzazione

Sviluppo concentrato in tre mesi (febbraio–aprile 2026), con iterazioni Agile e validazione progressiva su ambiente reale di laboratorio.

3.1 Metodologia di sviluppo

Il progetto è stato sviluppato nel periodo **febbraio–aprile 2026**, seguendo una metodologia **Agile** semplificata organizzata in sprint bisettimanali. Ogni sprint ha prodotto un incremento funzionante e verificabile, consentendo una prototipazione rapida e la raccolta precoce di feedback nell’arco di tre mesi di lavoro intensivo.

3.2 Fasi del progetto

Fase 1 – Analisi dei requisiti e design architetturale *Febbraio 2026*

Riunioni con il responsabile tecnico dell’istituto per definire quali laboratori monitorare, le soglie di allarme, la granularità temporale delle misure e i requisiti di disponibilità del sistema. Definizione dell’architettura hardware/software a quattro strati.

Fase 2 – Programmazione PLC e logiche di controllo *Febbraio – inizio Marzo 2026*

Sviluppo in TIA Portal dello schema Ladder: blocchi di acquisizione (FC10), elaborazione potenza (FC20), rilevamento anomalie (FC30) e comando distacco (FC40). Test in simulazione con S7-PLCSim.

Fase 3 – Servizio C# (s7.net e database) *Marzo 2026*

Implementazione della classe PLCDataService, sviluppo layer di accesso dati ADO.NET con query parametrizzate, creazione schema database e test end-to-end di lettura dal PLC e persistenza.

Fase 4 – Dashboard Electron *Marzo – Aprile 2026*

Sviluppo della dashboard Electron + Node.js con interfaccia HTML/CSS/JavaScript, aggiornamento real-time ogni 5 secondi, pannello allarmi e controllo distacco.

Fase 5 – Integrazione, test e installazione *Aprile 2026*

Setup del PLC e delle sonde presso gli impianti reali dei laboratori. Raccolta dati per due settimane. Verifica della coerenza tra misure PLC, database e dashboard.

Fase 6 – Documentazione e presentazione*Aprile 2026*

Redazione di questa relazione, preparazione di diagrammi, manuali operativi e materiale di presentazione.

Tabella 3.1: Pianificazione temporale del progetto Energy Monitor System (feb.–apr. 2026).

Fase	Descrizione	Inizio	Fine
Fase 1	Analisi requisiti e architettura	Febbraio 2026	Febbraio 2026
Fase 2	Programmazione PLC (TIA Portal)	Febbraio 2026	inizio Marzo 2026
Fase 3	Servizio C# e database	Marzo 2026	Marzo 2026
Fase 4	Dashboard Electron	Marzo 2026	Aprile 2026
Fase 5	Test integrazione e installazione	Aprile 2026	Aprile 2026
Fase 6	Documentazione e presentazione	Aprile 2026	Aprile 2026

4. Risultati Tecnici e Competenze Acquisite

I risultati mostrano un sistema funzionante e misurabile, con buone prestazioni, evidenza di impatto operativo e crescita concreta delle competenze digitali.

4.1 Funzionalità realizzate

Energy Monitor System è stato implementato completamente secondo le specifiche iniziali:

Sintesi per la commissione: il sistema è operativo su impianto reale, integra PLC industriale, backend C#, database e dashboard, e include funzioni di allarme e comando di distacco con tracciamento completo degli eventi.

- **Acquisizione dati real-time:** il PLC S7-1200 acquisisce i segnali da sonde multifunzione e memorizza in Datablock con periodo di aggiornamento di 100 ms.
- **Comunicazione S7:** il servizio C# legge i Datablock tramite s7.net con frequenza configurabile (default: ogni 5 secondi).
- **Persistenza database:** tutte le letture sono salvate con timestamp, consentendo storicizzazione e analisi retrospettiva.
- **Rilevamento anomalie:** il PLC confronta i consumi con soglie configurabili e genera allarmi; il servizio C# trasmette gli allarmi alla dashboard.
- **Dashboard interattiva:** visualizzazione real-time con grafici, storico, gestione allarmi.
- **Comandi di distacco:** operatore autorizzato può inviare comando di disconnessione per laboratorio; il PLC esegue il distacco e registra l'evento.

4.2 Metriche di prestazione

Durante il periodo di test (2 settimane), sono state raccolte le seguenti metriche:

4.3 Crescita delle competenze DigComp

La tabella seguente mappa le attività progettuali alle aree del framework **DigComp 2.2**:

Tabella 4.1: Metriche di prestazione e affidabilità del sistema.

Metrica	Valore	Target
Disponibilità sistema	99.7%	$\geq 99\%$
Latenza acquisizione PLC	100 ms	≤ 200 ms
Latenza lettura servizio C#	250 ms	≤ 500 ms
Latenza aggiornamento dashboard	5 s	≤ 10 s
Errori comunicazione S7 / settimana	3	≤ 5

Tabella 4.2: Mappa DigComp 2.2 – attività di Energy Monitor System.

Area DigComp	Livello	Attività nel progetto
Informazione e dati	6 – Esperto	Raccolta, elaborazione e storicizzazione di serie temporali energetiche
Creazione di contenuti	6 – Esperto	Sviluppo firmware PLC, servizio C#, query T-SQL, frontend Electron
Sicurezza	5 – Avanzato	Autenticazione operatore, autorizzazioni per comandi critici, log audit
Risoluzione di problemi	6 – Esperto	Debugging comunicazione S7, ottimizzazione query DB, tuning algoritmi PLC

4.4 Originalità e complessità tecnologica

Energy Monitor System si distingue per:

- **Stack tecnologico maturo:** integrazione di PLC industriale (S7-1200), protocolli proprietari Siemens (S7 Communication), accesso dati ADO.NET, framework front-end contemporaneo (Electron + Node.js + HTML/CSS/JavaScript).
- **Componenti multidisciplinari:** logica Ladder (mondo industriale/automazione), C# backend (mondo enterprise), JavaScript frontend (mondo web/app), database relazionale (mondo data).
- **Requisiti non-funzionali affrontati:** affidabilità (retry, timeout, graceful degradation), latenza (pool connessioni, caching), sicurezza (autenticazione, audit log), scalabilità (modularità architetturale).

4.5 Impatto energetico e operativo

I dati preliminari di monitoraggio hanno permesso di:

1. Identificare tre laboratori con profili di carico anomali fuori orario scolastico, suggerendo malfunzionamenti o cattive abitudini di shutdown.
2. Quantificare il consumo medio giornaliero per laboratorio, come baseline per valutare interventi di efficienza futura.
3. Validare l'algoritmo di rilevamento anomalie: il 92% degli allarmi rispecchiano reali picchi di assorbimento ($> 20\%$ sopra soglia).

5. Conclusioni e Prospettive

Il progetto conferma la fattibilità di soluzioni industriali avanzate in ambito IeFP e apre evoluzioni concrete su predizione, scalabilità e integrazione con altri sistemi.

5.1 Considerazioni conclusive

Energy Monitor System rappresenta un caso di studio completo su come affrontare un progetto di automazione industriale moderno: dall'analisi dei requisiti con il cliente, passando per la programmazione PLC e lo sviluppo backend, fino all'interfaccia utente. Gli studenti hanno sperimentato il ciclo di vita reale dello sviluppo software, affrontando sfide di integrazione, testing e deployment.

La scelta di utilizzare stack tecnologici attuali (S7-1200, s7.net, ADO.NET, Electron) dimostra che non è necessario scegliere tra *complessità industriale* e *modernità* per gli IeFP: questi due mondi possono coesistere e arricchirsi a vicenda.

Dal punto di vista pedagogico, il progetto ha confermato che l'apprendimento situato – cioè affrontare problemi reali con strumenti reali – produce una comprensione e una motivazione significativamente superiore rispetto all'insegnamento astratto.

5.2 Replicabilità e trasferibilità

L'architettura di Energy Monitor System è stata progettata pensando alla riproducibilità in contesti diversi:

- **Documentazione aperta:** codice sorgente (firmware PLC, servizio C#, dashboard Electron) rilasciato su GitHub con licenza MIT; datasheet e schemi disponibili.
- **Modularità:** ogni strato (PLC, acquisizione dati, dashboard) è indipendente. Un'istituzione potrebbe adottare solo il PLC + s7.net, oppure solo il frontend, senza obbligatorietà di usare tutta la stack.
- **Scalabilità:** il sistema supporta facilmente l'aggiunta di nuovi laboratori senza cambiamenti architetturali.
- **Portabilità database:** l'uso di ADO.NET con SQL standard e query parametrizzate consente l'adattamento a diverse piattaforme (SQL Server, PostgreSQL, SQLite) tramite driver dedicati.
- **Cross-platform UI:** Electron funziona su Windows, macOS, Linux.

5.3 Sviluppi futuri

Le seguenti evoluzioni rappresentano estensioni naturali del sistema:

1. **Machine learning per predizione:** addestramento di un modello su dati storici per prevedere picchi di consumo ore in anticipo, abilitando azioni preventive.
2. **Integrazione MQTT:** utilizzo di broker centralizzato per comunicazione publish/subscribe tra PLC e servizi, migliorando scalabilità e disaccoppiamento.
3. **Mobile app:** sviluppo di app nativa (MAUI) per accesso remoto alle letture e gestione allarmi da smartphone.
4. **Notifiche smart:** integrazione con sistema di notifiche scolastico (email, SMS, push desktop) per operatori.
5. **Certificazione energetica:** collegamento con organismi di certificazione per validare il risparmio energetico effettivo.
6. **Integrazione building automation:** collegamento con sistemi di controllo accessi, illuminazione domotica, clima per ottimizzazione globale.

5.4 Riflessione educativa

Questo progetto ha dimostrato che la didattica dell'automazione e dell'informatica presso gli istituti IeFP può raggiungere livelli di complessità e sofisticazione pari a corsi universitari, purché combinata con motivazione intrinseca (lavorare su problemi veri) e mentoring di qualità. Gli studenti Thomas Bravin, Miriana De Renzi e Jody Stabarin, attraverso questo percorso, hanno sviluppato non solo competenze tecniche trasferibili al mercato del lavoro, ma anche soft skill cruciali: ownership, problem-solving collaborativo, comunicazione tecnica.

A. Struttura del Datablock PLC S7-1200

Appendice tecnica con struttura dati PLC, utile per verifica, debugging e replicazione.

Il PLC memorizza i dati di consumo in un Datablock strutturato come segue:

```
1 // Datablock DB100: Energy Data
2 TYPE "EnergyData"
3   STRUCT
4     // Potenza istantanea per laboratorio (kW)
5     Lab1_Power : Real; // DWORD @ +0
6     Lab2_Power : Real; // DWORD @ +4
7     Lab3_Power : Real; // DWORD @ +8
8
9     // Energia cumulativa giornaliera (kWh)
10    Lab1_Energy : Real; // DWORD @ +12
11    Lab2_Energy : Real; // DWORD @ +16
12    Lab3_Energy : Real; // DWORD @ +20
13
14    // Flag allarmi
15    Lab1_Alarm : Bool; // Bit @ +24.0
16    Lab2_Alarm : Bool; // Bit @ +24.1
17    Lab3_Alarm : Bool; // Bit @ +24.2
18
19    // Timestamp ultimo aggiornamento (secondi da mezzanotte)
20    LastUpdate : UInt; // WORD @ +26
21
22    // Stato disconnessione
23    Disconnected : Bool; // Bit @ +28.0
24  END_STRUCT
25 END_TYPE
```

Listing A.1: Definizione della struttura Datablock DB100 in AWL/SCL.

B. Specifiche hardware

Elenco componenti principali impiegati, con ruoli e quantità per la replicazione del setup.

Tabella B.1: Componenti principali di Energy Monitor System.

Componente	Modello	Ruolo	Qt.
Automa programmabile	Siemens S7-1200 CPU 1214FC	Core PLC (safety non usata)	1
Modulo AI analogico	SM 1234 (4 canali, 12-bit)	Lettura sonde	1
Modulo DQ digitale	SM 1226 (8 uscite 24 VDC)	Comando distacco	1
Comm. Ethernet / TCP	CP 1243-1	S7 Communication	1
Sonde amperometriche	Modello in definizione	Misura corrente	3
Trasformatore TI / TT	100/5 A, 240/120 V	Adattamento segnali	3
Contattore trifase	Schneider LC1D25	Disconnessione lab.	3
Alimentatore 24 V / 5 A	Mean Well RSP-75-24	Alimentazione PLC	1

C. Accesso al repository GitHub

Riferimento al codice sorgente per trasparenza tecnica, verifica e riuso del progetto.

Il codice sorgente completo è disponibile presso:

<https://github.com/IAL-FVG-Pordenone/EMS-IAL>

Struttura del repository:

- `ConsoleDbPlc/`: progetto C# console per comunicazione PLC/database, con file `Program.cs`, `ConsoleDbPlc.csproj` e soluzione `ConsoleDbPlc.slnx`.
- `electron-fiddle-dashboard/`: dashboard Electron pubblicata con i file applicativi `index.html`, `main.js`, `renderer.js`, `preload.js`, `styles.css`, `db.js`, configurazione `db.config.example.json`, script SQL `Creazione_DB_EMS.sql` e file `package.json` / `package-lock.json`.
- `README.md`: descrizione generale del progetto e istruzioni essenziali.
- `LICENSE`: licenza MIT del repository.
- `.gitignore` e `.gitattributes`: configurazione del repository Git.

D. Contatti e supporto

Canali di contatto del referente per approfondimenti tecnici, demo o trasferimento del progetto.

Referente: Prof. Mauro Bortolani

Istituto: IAL Innovazione Apprendimento Lavoro FVG s.r.l.

Email: mauro.bortolani@ial.fvg.it